Redoing the Base Work and Identifying Gaps: Eligibility-Gated Greedy Dispatch versus Prediction-Aware Scheduling on Heterogeneous Cores

Lakshay S, Arkkadu Bharath Reddy

Reg. Nos. 26MAI0064, 26MAI0058 — Course: Computer Architecture and Organization

Guide: Dr. Yokesh Babu S, Associate Professor Grade 1. [Institution, department and e-mail to be filled by the authors]

***Abstract*—The base work of this project is US 11,561,826 B1 [1], which schedules the tasks of a dependency graph onto heterogeneous compute circuits through per-kernel task queues and threads, without predicting execution time. We *redo* the base work in the sense that is possible on a laptop: we re-read its claims, map its dispatch idea onto a reimplemented runtime (policy B4: eligibility-gated greedy dispatch), reimplement the classical list schedulers HEFT and PEFT [2], [3], and evaluate all of them on real executions of a Montage workflow and of image-processing task graphs on an Intel P/E-core laptop with a CUDA GPU. The redo shows where the base idea is strong (it keeps every worker busy at a cost of milliseconds) and where it fails: blind to task cost it needed 5% to 37% more time than a rule that merely knew which tasks were heavy, and on Montage it was 48% slower than static use of the P cores. From this redo and from the literature survey we derive ten gaps and propose a *prediction-aware, work-conserving, rank-ordered dispatcher* driven by a generic runtime model learned from execution logs, evaluated with overhead accounting and paired statistics against static HEFT and PEFT plans built from the same model. The proposed work has since been executed as a final benchmark (frozen protocol, ten Holm-adjusted contrasts, 24 included blocks on the primary workload, CPU and RTX 3050 GPU): the dynamic dispatcher is 17.8% faster than blind B4 on the CPU and 5.2% (CPU) to 12.2% (strict GPU set) faster than a static HEFT plan from the same model, but within 1% of a hand rule that knows the cost-relevant parameter, and equal to B4 where task cost does not vary. These measured limits delimit the contribution honestly.**

***Index Terms*—*base patent, reimplementation, heterogeneous scheduling, DAG, HEFT, runtime prediction, gap analysis, proposed work*

I. Introduction

Review 1 surveyed publications, patents and products on scheduling and offloading for heterogeneous resources and concluded that real-hardware evidence, overhead accounting, statistical reporting and informed baselines are missing. Review 2 has three aims: (i) *redo the base work*, i.e. rebuild and re-evaluate the dispatch concept of the base patent and of the classical list schedulers on our platform; (ii) *identify the gaps* that the redo and the survey expose; and (iii) state the *proposed work* that closes the gaps that can be closed on one laptop, leaving the Raspberry Pi stage out of scope (it was never measured). Since the first version of this review the proposed work has been executed as a final static-versus-dynamic benchmark; Section V-E gives its design and Review 3 its results.

We use the word "redo" carefully. The patent describes a hardware/software system for FPGA-based neural-network accelerators; a laptop cannot reproduce it. What can be redone is its *scheduling idea*. An earlier project note recorded that B4 is a comparison baseline and not an implementation of the patent; this paper keeps that rule and adds what was missing: a mapping at the level of the granted claims, which we have now read, and a re-evaluation of the baseline on real workloads.

II. Base Work

A. The base patent

US 11,561,826 B1 [1] was filed by Xilinx on 12 Nov. 2020 and granted on 24 Jan. 2023 with 20 claims. Claim 1 recites instantiating kernel objects of kernel types that indicate compute circuits; generating a graph whose nodes are tasks assigned to kernel objects and whose edges are data dependencies; creating task queues and assigning kernel objects and tasks to them; enqueueing tasks by threads executing the kernel objects; and dequeueing tasks and activating the compute circuits. Dependent claims add blocking and non-blocking kernels (2), several graphs (3), shared queues (4–5), multi-threaded kernels with a configurable thread count (6–7), performance measurement that adjusts the thread count (8), output buffers and in-degree counters (9–10), utilisation-triggered scaling (11), a report function (12) and concurrent graphs (13); claims 14–20 restate the method as a system. No claim recites prediction of execution time, a transfer-cost model, or offloading to a remote device.

B. Base papers

The 12 supplied papers [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15] define the research setting: DAG scheduling and dependent-task offloading with reinforcement learning, mostly in simulation. The closest is LACHESIS [4]: a GNN selects the next task and a duplication-based rule places it. We did not re-run these learned systems, because their code, simulators and training budgets are not part of the supplied material, and we say so; instead we redo the two *classical* algorithms that most of them use as baselines, HEFT and PEFT, and compare the learned-prediction approach against them on real executions.

III. Redoing the Base Work

A. Claim-level mapping

TABLE I

BASE PATENT CONCEPTS AND THEIR STATUS IN OUR RUNTIME

Patent element (claims)	Our runtime	Status
Kernel object of a kernel type = compute circuit (1, 14)	Worker pools P, E, CUDA, remote; per-task eligible pools	Mapped
Graph: nodes = tasks, edges = data dependencies (1)	DAG with dependency counters for every policy	Mapped (all policies)
Task queues per kernel; kernels assigned to queues (1)	One global FIFO ready queue scanned for eligibility	Approximated
Threads enqueue/dequeue by assignment (1)	Idle pinned worker process receives oldest eligible ready task	Mapped (push model)
In-degree counters decremented on completion (9–10)	Readiness when all parents completed	Mapped
Blocking / non-blocking kernels (2, 15)	Every task is a synchronous process call	Not implemented
Several graphs, shared queues (3, 4–5, 16–18)	One DAG per run	Not implemented
Performance measurement adjusts thread count (8, 11)	Fixed one worker per core; utilisation logged, not acted on	Not implemented
Report function with kernel statistics (12)	Event log, run manifest, dashboard	Partly
FPGA compute circuits	CPU P/E pools and a CUDA GPU worker	Different hardware

The core of the mapping is the line between rows two and four: **a worker takes a task only if the task is declared eligible for it, and the choice among eligible idle workers uses no estimate of execution time.** We implement it as B4 with priority P, then E, then worker identifier. Its decision cost is therefore very small, which we measured (Section III-C).

B. Experimental setting of the redo

All results are measured on one laptop (Intel Core i5-13500HX with 6 P and 8 E cores, RTX 3050 6 GB, 15.7 GB RAM, native Windows 11); workers are pinned processes, the coordinator has a reserved core, outputs are verified against a CPU reference, and conditions are run in interleaved, seeded blocks with a memory and power preflight. The effect of a policy is the paired geometric mean change of makespan against a reference within blocks with a percentile bootstrap interval (negative = faster). Workloads: the 59-task Montage mosaic (nine 160 s reprojection tasks, 36 difference fits; native Windows build, three identical sequential reference runs of 1459–1469 s); a uniform batch of 64 identical image chains; and *hidden-cost* batches in which 4 to 40 of 64 chains are 60 times heavier in one parameter. HEFT and PEFT [2], [3] were reimplemented and reproduce the published worked examples (see Section III-D).

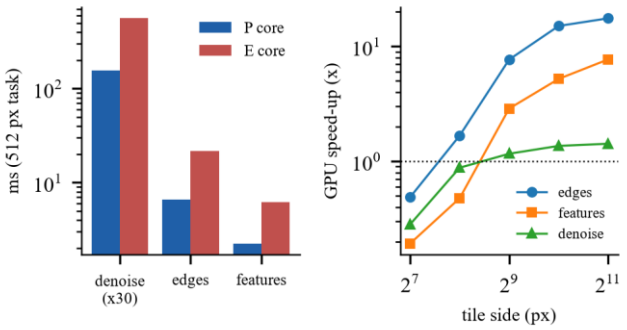


Fig. 1. Measured heterogeneity of the redo platform: kernel time of three stages on a P and an E core (left) and GPU speed-up versus tile size (right).

C. Baselines, estimators and the effect measure

TABLE II

POLICIES AND ESTIMATORS OF THE REDONE RUNTIME

Name	Rule	Uses prediction?
B0	static: all tasks on the P cores (or E cores)	no
B1 / B3	round-robin / shortest queue over eligible workers	no
B2	fixed task-type to pool map	no
B4	first idle eligible worker, priority P, E (the patent-mapped base idea)	no
B5	B4 with the ready queue ordered by bottom level (criticality)	structure only
EFT (E0–E4)	earliest finish time = wait + transfer + execution; E0 constant, E1 solo profile, E2 analytic size model, E3 moving history, E4 ridge on occupancy	yes
HEFT, PEFT	static list plans (upward rank / optimistic cost table)	yes (cost matrix)
Hand rule	B4 with the ready queue sorted by the cost-relevant task parameter	no (knowledge)
Proposed	rank-ordered, idle-worker-only dispatch with a learned generic model	yes

The effect of a policy c against a reference r is computed per block b as $\delta_b = \ln(M_{c,b} / M_{r,b})$, where M is the makespan; the reported effect is $(\exp(\text{mean } \delta_b) - 1) \times 100\%$, with a percentile bootstrap interval stratified by workload instance and a sign-flip p-value adjusted by the Holm procedure over the confirmatory family. Equivalence claims use a ±2% margin on the interval.

D. Results of the redo

TABLE III

REDO OF THE BASE DISPATCH IDEA (B4) AND CLASSICAL LIST SCHEDULERS; PERCENTAGES ARE PAIRED MAKESPAN CHANGES WITH 95% INTERVALS (NEGATIVE = SHORTER)

Workload (blocks)	Comparison	Result	Reading
Montage, 59 tasks, 6P+7E (9)	B4 vs static P-only	+47.87% [+37.5, +54.6]	greedy use of slow E cores hurts
Montage	EFT, HEFT vs static P-only	+1.62%, +1.54% (intervals above 0)	no headroom: every task prefers P
Uniform batch + GPU (30)	static P-only vs B4	+35.9% [+34.8, +36.9]	B4 uses all workers: 26% shorter
Hidden cost, 16 heavy, 6P+7E (36)	hand rule vs B4	−19.85% [−20.7, −18.9]	B4 blind to cost
Hidden cost, 16 heavy, 6P+7E (24, 6 orders)	HEFT, PEFT vs B4	−18.08%; −2.58%	HEFT strong; PEFT weak
Hidden cost, 4 heavy (12)	hand rule vs B4; static P-only vs B4	−26.94%; −11.56%	B4 worse than P-only
Hidden cost, 40 heavy (12)	hand rule vs B4	−5.15% [−6.2, −4.0]	gain shrinks
Two-factor batch (40)	hand rules vs B4	−0.47%, +1.98% (n.s.)	ties B4

B4 is a good baseline for keeping workers busy. On the uniform batch it is 26% shorter than static use of the P cores, because it uses idle E workers and the GPU. Its decision cost is small: 7 ms of per-dispatch decision time per run for 257 tasks, 0.2 s of coordinator time (about 5% of a 4 s run). **B4 is a poor baseline when task cost differs and is invisible to the scheduler.** With 4 or 8 heavy chains among 64, B4 was 37% slower than the informed rule and even slower than P-only, because it hands a heavy chain to whichever worker is free first, including a slow E core, or starts it late. On Montage the result is the same in a different guise: every task prefers P cores by a factor of 2.3 to 3.6, so the only way to lose is to use E cores, and B4 loses 48%.

Why the base idea cannot do better. Event logs of 20,046 B4 dispatches on the GPU workload show that 93.6% had exactly one eligible idle worker; only 6.4% offered a choice. An eligibility-gated dispatcher chooses the *order* implicitly (first come, first served) and the *worker* almost never. Order is exactly what the base idea does not control, and Section V shows that order is where the available gain lies.

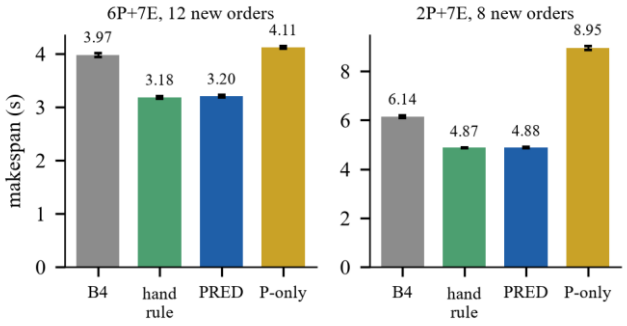


Fig. 2. Mean makespan per policy on the hidden-cost workload with 12 (left) and 8 (right) chain orders not used before; bars show geometric means with 95% intervals. B4 is the redone base idea.

E. Redo of HEFT and PEFT

HEFT ranks tasks by upward rank, $rank_u(n_i) = \bar{w}_i + \max_{*n_j* \in succ(*n_i*)} (\bar{c}_{ij} + rank_u(n_j))$, and assigns each task, in rank order, to the processor with the earliest finish time using insertion into idle gaps [2]. PEFT replaces the rank by the mean of an *optimistic cost table*, $OCT(t_i, p_k) = \max_{*t_j* \in succ(*t_i*)} \min_{*p_w*} [OCT(t_j, p_w) + w(t_j, p_w) + \bar{c}_{ij}]$ (with $\bar{c}_{ij} = 0$ when $p_w = p_k$), and picks the processor minimising the finish time plus *OCT* [3]. We implemented both as *static plans* from a cost matrix and, to compare like with like, fed them the same learned runtime model as our dispatcher. Both implementations reproduce the published worked examples through the repository code (the 10-task example of the HEFT paper, and the PEFT sample as printed in the author's thesis chapter, where PEFT yields schedule length 122 and HEFT 133; the only tolerated difference is the tie-breaking of two equal ranks); the IEEE article of PEFT itself was not read. On 24 blocks of six unseen chain orders HEFT was 18.1% and PEFT 2.6% shorter than B4, against 19.0% for the proposed dispatcher; HEFT versus the dispatcher was +1.1% with an interval of [−0.5, +2.8] (not distinguishable), PEFT +20.2%.

F. Root cause of the base idea's losses

Two mechanisms explain B4's losses. *Late start:* with a shuffled ready queue a heavy chain may be dispatched near the end of the run, so the makespan equals the time until it starts plus its own duration; the damage depends on the chain order (B4's mean makespan varied by 0.27 s between two orders while the learned dispatcher's did not) and is largest when few chains are heavy (Fig. 4). *Wrong core:* when all P workers are busy B4 gives the next task to an idle E worker, which is 2.8 to 3.6 times slower for these stages. A scheduler that knows or predicts task cost fixes both by serving heavy tasks first, which is why an informed hand rule and the learned dispatcher achieve the same gain.

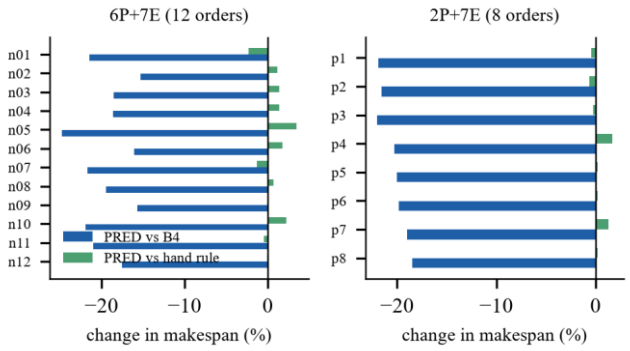


Fig. 3. Per-order change of makespan of the proposed dispatcher against B4 (blue) and against the informed hand rule (green) for 12 orders (left) and 8 orders (right) not used before.

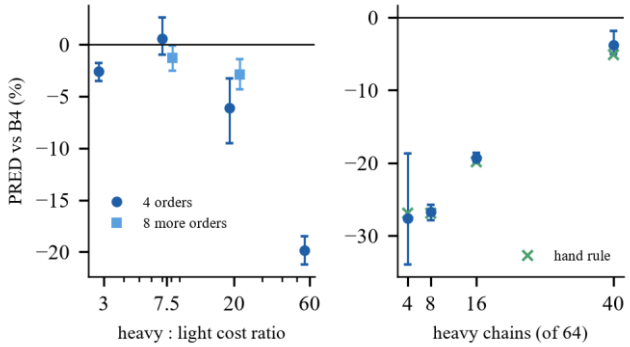


Fig. 4. Gain of the proposed dispatcher over B4 as the heavy:light cost ratio grows (left; 4 and 8 additional orders) and as the number of heavy chains among 64 changes (right).

G. A defect found while redoing the baseline

Instrumenting B4 explained a discrepancy between two overhead figures. The coordinator's total scheduling time per run (0.19 s) was 28 times the sum of the per-dispatch decision times (7 ms). In one instrumented run B4 made 10,299 calls of its worker-selection function and 10,042 of them returned nothing: the worker pool always contained an idle CUDA worker that no task could use, so the dispatch loop never reached its early exit and scanned every ready task on every pass. A decision-preserving fix, tested in a separate adapter, cut coordinator time to 8 ms for B4 and 24 ms for the dispatcher and changed the dispatcher's gain from 19.5% to 20.3% over B4, i.e. 0.8 percentage points. This is a reminder that any baseline redone on new infrastructure must be instrumented, not only timed.

H. Threats to the validity of the redo

- B4 is the patent's *idea* mapped to pools, not its FPGA system; no statement about the patent's performance follows.
- HEFT and PEFT are static plans without re-planning and follow the formulas of the papers' worked examples; the learned systems of the base papers were not rerun.
- One machine; runs took place with a dirty working tree and several source hashes, so sessions are never pooled in an analysis.
- Workloads were designed to contain a hidden cost; null workloads (Montage, uniform, two-factor) are reported with the same weight.

IV. Gap Identification

Table IV merges the evidence of the survey (Review 1) with the evidence of the redo. A gap is listed only if it is supported by both a source and a measurement or a missing feature that we verified.

TABLE IV

Gaps identified and the contribution each motivates

Gap	Evidence	Addressed by the proposed work?
G1 Real-hardware evidence is rare	2 of 13 papers use physical devices; the base patent targets FPGAs	Yes: P/E cores, CUDA GPU, real Montage; Pi later
G2 Scheduler overhead rarely counted	decision time in 4 of 13 papers; our coordinator time is 2–5% of a run	Yes: two overhead metrics per run
G3 No interval estimates	no "confidence interval" in 12 papers	Yes: paired bootstrap, Holm, pre-registration
G4 Missing informed baselines	no paper tests a rule that knows the hidden cost	Yes: hand rules, HEFT, PEFT
G5 Base idea is cost-blind	B4 5–37% slower than an informed rule	Yes: predicted-rank ordering
G6 Eligibility is static	93.6% of dispatches have one idle eligible worker	Partly: order, not worker choice, is exploited
G7 Prediction accuracy ≠ scheduling value	accurate remote predictions gave no gain (emulation)	Yes: reported separately
G8 Generalisation untested	one instance per setting is common	Yes: 20 unseen orders, dose and heavy-count sweeps
G9 Remote execution only simulated	no real-network offloading result in the base set	Prepared; emulation only, no Raspberry Pi or real-network result
G10 Static plan vs dynamic dispatch not compared end to end	HEFT is a baseline in 5 of 13 papers, all in simulation; earlier session: HEFT level with the dispatcher (+1.10%, old timing convention)	Yes: final benchmark, one cost model on both sides, end-to-end timing, CPU and GPU

V. Proposed Work

A. Objective and hypotheses

Objective. Determine, by measurement on real heterogeneous hardware and with the scheduler's own cost counted, when execution-time prediction improves DAG scheduling over (a) the base idea B4, (b) static placement, (c) informed simple rules, and (d) HEFT/PEFT. The proposal is not a new learning algorithm and not a claim that prediction always wins. Falsifiable hypotheses, written before the experiments:

1) **H1** A rank-ordered, work-conserving dispatcher that uses predicted costs shortens makespan relative to B4 when task cost is hidden from size-keyed tables and several tasks compete for few fast workers.
2) **H2** The dispatcher is not distinguishable from an informed rule that knows the cost-relevant parameter (it replaces that knowledge, it does not exceed it).

3) **H3** Where no cost is hidden or no order can be changed, the dispatcher is equivalent to B4 within ±2% and does no harm.
4) **H4** Accurate predictions are not sufficient for gain: offloading to a remote worker pays only if transfer cost is small relative to compute and the remote worker adds capacity.
5) **H5** With the *same* cost model on both sides, the dynamic dispatcher is not slower than a static HEFT plan on the CPU and is faster when a CUDA worker is added (stated before the data: CPU result inconclusive or equivalent).
6) **H6** Using the GPU only for stages whose speed-up was verified helps the model-driven dispatcher, but does not help blind B4.
7) **H7** A hybrid selector between B4 and the dispatcher is worth building only if its headroom exceeds the ±2% margin in the development cells.

B. Architecture of the proposal

The proposal extends the redone runtime (Fig. 5): the coordinator keeps the dependency gating of the base idea and adds an estimator and a rank-ordered, idle-worker-only policy. A remote worker is reached through a proxy and a node process; the link is a delay shim for emulation or the real network for a Raspberry Pi. Every run writes an event log and a manifest with code and environment hashes.

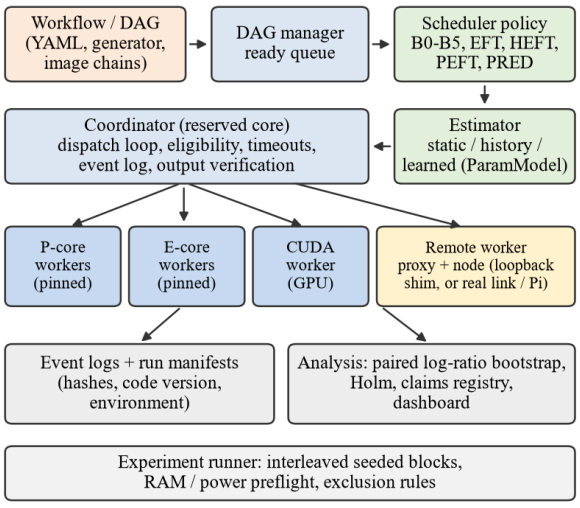


Fig. 5. Architecture of the proposed system built on the redone runtime.

C. Algorithm: predicted-rank, work-conserving dispatch

Let $\hat{y}_{s,p}(x)$ be the predicted kernel time of a task of stage s with numeric parameters x on pool p. The model is fitted per (stage, pool) from earlier execution logs by ridge regression on the parameters that actually vary in the logs and on their pairwise products; no stage or parameter name is coded. The rank of a task is its predicted cost plus the largest rank of its successors (HEFT-style upward rank):

$$\text{rank}(t) = \hat{y}(t) + \max_{c \in \text{succ}(t)} \text{rank}(c) \quad (1)$$

At every dispatch pass the coordinator proceeds as follows:

1) order the ready tasks by decreasing rank (ties first-come first-served);
2) for each ready task t in that order, consider only *idle* workers eligible for t; if none exists, leave t queued (no early binding to a busy worker);
3) send t to the idle eligible worker w minimising $\hat{y}$ on the pool of w;
4) on completion, log the observed kernel time; an offline step refits the model from logs (one command, no code change for a new pool layout).

Work conservation (step 2) and late binding are what distinguish it from earlier early-binding EFT variants that lost about 6% to a hand rule in preliminary runs; rank ordering (step 1) is the part that removes B4's late start of heavy tasks. The model's inference is a dot product of a few numbers, and the measured per-dispatch decision time is 15 ms per 257-task run.

D. Evaluation methodology

- **Workloads:** Montage (real executables), uniform and hidden-cost image-chain batches, a contended mixed DAG, an image pipeline with an emulated remote pool; later a Raspberry Pi 5 worker.
- **Baselines:** B0 static P-only, B4 (base idea), informed hand rules, EFT ladder, HEFT, PEFT, an oracle replay used only as a reference.
- **Metrics:** makespan (primary), per-dispatch decision time, coordinator scheduling time, queue wait, transfer time, task time, output correctness.
- **Statistics:** paired log-ratio of makespan within interleaved blocks, percentile bootstrap, Holm correction over the confirmatory family, equivalence margin of ±2% for no-harm claims, designs and amendments written before runs, a claims registry re-checked against stored tables by tests.
- **Integrity:** memory and power preflight, one pool per session, exclusion rules written beforehand, failed runs kept and flagged, SHA-256 manifests, reproduction of tables from raw logs.

E. Final benchmark design (executed)

The final benchmark fixes the protocol before the evaluation sessions. *Static* policies compute a complete plan before the run: HEFT and PEFT with the learned model (HEFT_param_model, PEFT_param_model), HEFT with end-to-end cost terms (HEFT_E2E), and an oracle plan from class-median costs used only as a reference. *Dynamic* policies de-

cide when a worker is idle: B4, B4 with the queue sorted by the cost parameter (B4_param_first), the predicted-rank dispatcher (PRED_RANK_GREEDY) and a variant that minimises predicted occupancy, transfer plus kernel plus set-up plus inter-process time (PRED_RANK_GREEDY_E2E). Hardware sets are CPU only (6 P and 7 E workers), a strict GPU set (the CUDA worker takes only stages whose speed-up was verified on the original spike record) and an extended set (denoise at further verified iteration counts). The primary workload has 16 heavy chains of 64; five further workloads vary the heavy count, the cost ratio and the uniform case. The end-to-end makespan runs from the start of the run to the last completion, so start-up and policy set-up are inside the number (amendment F1). Ten contrasts on the primary session (C1–C10) form the confirmatory family under Holm correction; everything else is exploratory.

TABLE V

CONFIRMATORY CONTRASTS FIXED IN THE PROTOCOL AND THEIR OUTCOME (PRIMARY WORKLOAD, 24 BLOCKS)

Id	Contrast	Effect % [95% CI]	Result
C1	PRED vs B4 (CPU)	−17.8 [−18.8, −16.7]	faster
C2	PRED-E2E vs PRED (CPU)	+0.6 [+0.1, +1.3]	equivalent
C3	PRED vs HEFT (CPU)	−5.2 [−5.9, −4.6]	faster
C4	PRED-E2E vs HEFT-E2E (CPU)	−5.2 [−6.0, −4.5]	faster
C5	PRED vs HEFT (strict GPU)	−12.2 [−13.4, −11.1]	faster
C6	PRED-E2E vs HEFT-E2E (strict GPU)	−8.5 [−9.7, −7.6]	faster
C7	PRED-E2E vs PRED (strict GPU)	−0.0 [−0.7, +0.5]	equivalent
C8	PRED vs B4 (strict GPU)	−21.2 [−22.6, −19.7]	faster
C9	PRED strict GPU vs PRED CPU	−2.6 [−3.3, −1.7]	faster
C10	B4 strict GPU vs B4 CPU	+1.6 [+0.2, +3.0]	inconclusive

F. Requirements derived from the gaps

TABLE VI

REQUIREMENTS, THE GAP EACH ANSWERS, AND HOW IT IS VERIFIED

Requirement	Gap	Verification
R1 run real task graphs on pinned P/E workers and a CUDA worker	G1	event logs record worker, pool, start and end of every task
R2 verify every output against a CPU reference	G1	hash equality or stated tolerance; runs with mismatch are flagged
R3 log two overhead metrics per run	G2	per-dispatch decision time and coordinator scheduling time
R4 interleaved, seeded blocks with preflight and exclusion rules	G3, G8	design file hash stored in every run manifest
R5 include informed rules, HEFT and PEFT	G4	policies in one registry with eligibility enforced for all
R6 learn costs from logs without hard-coded stage names	G5	tests: unseen stage or pool raises instead of guessing
R7 keep prediction accuracy and scheduling value separate	G7	separate offline and end-to-end analyses
R8 provide an external-node path for a Raspberry Pi	G9	loopback rehearsal of the full command chain (25 runs ok)

G. Run protocol and data flow

One run builds the DAG from the workflow description, creates a fresh work directory, stages the inputs, builds the policy and estimator named by the condition, executes the graph on the shared worker pool and verifies the outputs. The coordinator writes one event per readiness, dispatch, submission, completion and failure, with the decision time attached to each dispatch. The experiment runner derives the block plan from the design seed, runs every condition once per block in random order, repeats the memory (and mains-power) preflight before every block, and writes a manifest per run with the code hash, environment hash, condition, seed, status and output hashes. Warm-up blocks are excluded by the only rule the design allows; failed or unverified runs stay in the raw data and are excluded from the primary analysis by a rule fixed in advance, with a sensitivity analysis that includes them. The analysis re-derives every table from the raw logs, and a registry test re-reads each cited number from its stored table.

H. Positioning against the closest work

Three works are closest. IBM's grant [16] ranks DAG tasks with a learning agent and dispatches in rank order; our dispatcher differs by using a generic, parameter-discovering regression model, placing only on idle workers, and being evaluated against informed rules with intervals, but the overlap in *rank-ordered dispatch* is real and must be assessed by a patent professional before any filing. Lotaru [17] and StarPU [18] supply runtime predictions to HEFT-type or StarPU schedulers; we use the same idea with a lighter model and test it on real executions, including the cases where it does not help. LACHESIS [4] is the strongest learned DAG scheduler; we do not compete with it on simulated clusters and instead ask when a much cheaper model is enough.

I. Differentiation from the base patent and papers

TABLE VII

WHAT IS NEW RELATIVE TO THE BASE WORK

Aspect	Base patent / papers	Proposed work
Execution-time prediction	none in claims; learned policies in papers	generic log-learned model; decision overhead measured
Ordering of ready tasks	FIFO per kernel queue	predicted upward rank
Worker choice	eligibility + availability	idle eligible worker with least predicted time
Evaluation	FPGA system (patent); simulation (papers)	real laptop runs; paired intervals; unseen instances
Baselines	none stated; random/FIFO/HEFT	adds informed hand rules and B4 as the base idea
Claim scope	system architecture	empirical study of when prediction helps

J. Expected contributions, novelty and risks

We claim a measured methodology and map, not a new algorithm. Rank-ordered dispatch with a learning agent is disclosed in a granted IBM patent [16] and HEFT-type ranks are classical, so novelty is limited to the combination, the generic parameter-discovering model, the work-conserving rule, and above all the evidence. Main risks: the headline gain comes

from a workload that contains a designed hidden cost (mitigated by the no-harm and null workloads and by reporting that an informed rule performs equally); learned models were fitted on one machine; and the Raspberry Pi has not yet been measured.

K. Results to date and plan

Measurements (Review 3 gives details) support H1–H3: on twenty chain orders not used before the dispatcher was 19.3% (6P+7E) and 20.5% (2P+7E) shorter than B4, within ±1.6% of the informed rule, and within ±2% of B4 on the uniform batch. H4 was supported in emulation. The final benchmark adds H5–H7: the dispatcher is faster than a static HEFT plan from the same model (5.2% on the CPU, 12.2% with the strict GPU set), the GPU helps the dispatcher by 2.6% (strict set) to 7.5% (extended set) but not blind B4, and a hybrid selector is not justified. Two results limit the claim: a hand rule that sorts by the cost parameter is within 1% of the dispatcher in every configuration, and the earlier overnight session found HEFT level with the dispatcher (+1.10%, different orders and timing convention); the difference was not isolated. The remaining plan: (1) finish the paper drafts; (2) a second workload family; (3) publish code and data after the IPR decision. The Raspberry Pi stage is out of scope.

TABLE VIII

HYPOTHESES, THE EXPERIMENT THAT TESTS EACH, AND THE PRELIMINARY OUTCOME

Hyp.	**Test**	**Preliminary outcome (exploratory sessions)**
H1 gain over B4	hidden-cost batches, 6P+7E and 2P+7E, unseen orders	−19.3% [−20.0, −18.6] (12 orders); −20.5% [−21.2, −19.8] (8 orders)
H2 equal to informed rule	same sessions, dispatcher vs hand rule	+0.66% [−0.2, +1.5]; +0.27% [−0.2, +0.8]
H3 no harm	uniform batch; two-factor batch; Montage	uniform +0.45% [−0.3, +1.2]; two-factor +0.95% [−0.9, +2.8]; Montage no gain
H4 accuracy is not value	emulated remote pool, denoise level 30	predicted vs measured remote time 0.1457 vs 0.1464 s; remote worker made EFT 3.8% slower
H5 dynamic vs static, same model	C3, C5 (primary workload, 24 blocks)	−5.2% [−5.9, −4.6] CPU; −12.2% [−13.4, −11.1] strict GPU; stated expectation for the CPU contrast (inconclusive or equivalent) was wrong
H6 GPU where verified	C9, C10	PRED −2.6% [−3.3, −1.7]; B4 +1.6% [+0.2, +3.0], inconclusive
H7 hybrid selector	headroom in 17 development cells	maximum 1.1%, none above 2%: not justified, not built

TABLE IX

WORK PLAN AND STATUS

Work package	**Status**
Runtime, estimators, policies, experiment framework, dashboard	done; 521 automated tests pass
Montage native build, reference runs, adapter	done; null scheduling result reported
Hidden-cost study, unseen orders, dose and heavy-count sweeps	done (exploratory)
Overhead diagnosis and dispatch-loop A/B	done; source fix proposed, not applied
Remote worker emulation, external-node path, Pi runbook	done; Raspberry Pi not yet measured
Final static-vs-dynamic benchmark (calibration, holdout, six workloads, CPU and GPU)	done; confirmatory family reported
Raspberry Pi 5 real-link experiment	out of scope; emulation only
Second workload family; clean-tree rerun of the headline result	planned

VI. CONCLUSION

Redoing the base work showed that eligibility-gated greedy dispatch is an economical way to keep heterogeneous workers busy but is blind to task cost, and that this blindness costs between 5% and 37% on cost-skewed workloads and 48% on Montage relative to the best simple alternative. Ten gaps were identified and a prediction-aware, work-conserving, rank-ordered dispatcher with overhead accounting and paired statistics was proposed. Its measured benefit is real but conditional, which is what the proposed study set out to quantify: it beats blind dispatch by about 18% and a static plan from the same model by 2% to 12% when task cost varies, and a simple rule that knows the cost parameter gets almost the same result.

REFERENCES

[1] S. Nagpal, A. Karumannil, V. Jain, and A. K. Patil (Xilinx, Inc.), "Scheduling processing of machine learning tasks on heterogeneous compute circuits," U.S. Patent 11,561,826 B1, Jan. 24, 2023 (filed Nov. 12, 2020).

[2] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 260–274, Mar. 2002.

[3] H. Arabnejad and J. G. Barbosa, "List scheduling algorithm for heterogeneous systems by an optimistic cost table," *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 3, pp. 682–694, Mar. 2014, doi: 10.1109/TPDS.2013.57.

[4] J. Luo, Y. Zhou, X. Li, M. Yuan, J. Zeng, and J. Yao, "Learning to optimize DAG scheduling in heterogeneous environment," in *Proc. IEEE Int. Conf. Mobile Data Manage. (MDM)*, 2022, doi: 10.1109/MDM55031.2022.00040 (arXiv:2103.06980).

[5] C. Sun, T. Yang, and Y. Lei, "DDDQN-TS: A task scheduling and load balancing method based on optimized deep reinforcement learning in heterogeneous computing environment," *Int. J. Intell. Syst.*, vol. 37, no. 11, pp. 9138–9172, 2022, doi: 10.1002/int.22983.

[6] J. Wang, J. Hu, G. Min, W. Zhan, A. Y. Zomaya, and N. Georgalas, "Dependent task offloading for edge computing based on deep reinforcement learning," *IEEE Trans. Comput.*, 2022, doi: 10.1109/TC.2021.3131040.

[7] A. Rodríguez, A. Navarro, K. Nikov, J. Nunez-Yanez, R. Gran, D. Suárez Gracia, and R. Asenjo, "Lightweight asynchronous scheduling in heterogeneous reconfigurable systems," *J. Syst. Archit.*, vol. 124, Art. no. 102398, 2022, doi: 10.1016/j.sysarc.2022.102398.

[8] Y. Song, C. Li, L. Tian, and H. Song, "A reinforcement learning based job scheduling algorithm for heterogeneous computing environment," *Comput. Electr. Eng.*, vol. 107, Art. no. 108653, 2023, doi: 10.1016/j.compeleceng.2023.108653.

[9] B. Gong and X. Jiang, "Dependent task-offloading strategy based on deep reinforcement learning in mobile edge computing," *Wireless Commun. Mobile Comput.*, 2023, doi: 10.1155/2023/4665067.

[10] J. Liu, Y. Mi, X. Zhang, and X. Li, "Task graph offloading via deep reinforcement learning in mobile edge computing," *Future Gener. Comput. Syst.*, vol. 158, pp. 545–555, 2024, doi: 10.1016/j.future.2024.04.034 (arXiv:2309.10569).

[11] X. Chen, J. Cao, Y. Sahni, S. Jiang, and Z. Liang, "Dynamic task offloading in edge computing based on dependency-aware reinforcement learning," *IEEE Trans. Cloud Comput.*, vol. 12, no. 2, pp. 594–608, Apr.–Jun. 2024, doi: 10.1109/TCC.2024.3381646.

[12] J. Liu, P. Xie, W. Li, B. Tang, and J. Liu, "Dynamic task offloading scheme for edge computing via meta-reinforcement learning," *Comput. Mater. Contin.*, vol. 82, no. 2, 2025, doi: 10.32604/cmc.2024.058810.

[13] C. Liu, H. Wang, M. Zhao, J. Liu, X. Zhao, and P. Yuan, "Dependency-aware online task offloading based on deep reinforcement learning for IoV," *J. Cloud Comput.*, vol. 13, Art. no. 136, 2024, doi: 10.1186/s13677-024-00701-0.

[14] G. Nieto, I. de la Iglesia, U. Lopez-Novoa, and C. Perfecto, "Deep reinforcement learning techniques for dynamic task offloading in the 5G edge-cloud continuum," *J. Cloud Comput.*, vol. 13, Art. no. 94, 2024, doi: 10.1186/s13677-024-00658-0.

[15] Q. Tan, W. Chen, and D. Liu, "Task scheduling for heterogeneous multicore processors based on deep reinforcement learning," *Int. J. Intell. Syst.*, vol. 2025, Art. no. 7562400, 2025, doi: 10.1155/int/7562400.

[16] A. Vega *et al.* (International Business Machines Corp.), "Heterogeneous system on a chip scheduler with learning agent," U.S. Patent 11,740,933 B2, Aug. 29, 2023.

[17] J. Bader, F. Lehmann, L. Thamsen, U. Leser, and O. Kao, "Lotaru: Locally predicting workflow task runtimes for resource management on heterogeneous infrastructures," *Future Gener. Comput. Syst.*, vol. 150, pp. 171–185, 2024, doi: 10.1016/j.future.2023.08.022.

[18] C. Augonnet, S. Thibault, R. Namyst, and P.-A. Wacrenier, "StarPU: A unified platform for task scheduling on heterogeneous multicore architectures," *Concurrency Comput.: Pract. Exper.*, 2011 (journal version; see also Euro-Par 2009, LNCS 5704, pp. 863–874).